EustressEngine

Pure Rust Optimization Implementation Guide

eustress/crates/engine · eustress/crates/common · eustress-baker

Targeting: Radiance Cascades GI · Meshoptimizer Pipeline · GPU-Driven Rendering · ECS Throughput

How to use this document

Each section follows the same structure: priority badge (red = must-have, amber = high value, green = nice-to-have), explanation, cargo dependencies, and copy-paste Rust code. Sections are ordered by implementation dependency — complete them roughly top to bottom. GPU sections assume wgpu 0.20+, ECS sections assume Bevy 0.14.

Contents

Part 1 — Asset Baker Pipeline (offline, separate binary)

Part 2 — GPU Optimizations (wgpu / render graph)

Part 3 — CPU / ECS Optimizations (Bevy)

Part 4 — World State Optimizations

Part 5 — Radiance Cascades Global Illumination

Part 6 — Cargo workspace setup

Part 1 — Asset Baker Pipeline

The baker is a separate workspace binary (eustress-baker) that runs offline. It reads glTF/FBX source assets, runs the full meshoptimizer pipeline, and writes proprietary .emesh binary files. The engine runtime only reads .emesh — it never processes raw source assets.

1.1 Workspace binary setup

MUST-HAVE

Add eustress-baker as a workspace member. It links meshopt at compile time but the engine runtime only links the decode half.

```
# eustress/Cargo.toml  (workspace root)
[workspace]
members = [
  "crates/engine",
  "crates/common",
  "crates/client",
  "crates/baker",     # new
]

# crates/baker/Cargo.toml
[package]
name = "eustress-baker"
version = "0.1.0"

[dependencies]
meshopt  = "0.3"           # meshoptimizer Rust bindings
gltf     = "1.4"           # parse Blender glTF exports
zstd     = "0.13"          # two-pass compression layer
notify   = "6"             # file-change watcher for hot-reload
glam     = "0.27"          # shared math with engine
bytemuck = { version="1", features=["derive"] }
serde    = { version="1", features=["derive"] }
bincode  = "1"

# crates/engine/Cargo.toml  (runtime — decode only)
[dependencies]
meshopt  = { version="0.3", default-features=false, features=["decode"] }
```

1.2 Vertex deduplication — Stage 1

MUST-HAVE

Raw glTF exporters produce duplicate vertices because modelling tools store position, normal, and UV as separate indexed streams. This step merges them into a single unique-vertex index buffer. Without it every downstream algorithm works on inflated data.

```
use meshopt::{generate_vertex_remap, remap_index_buffer, remap_vertex_buffer};

pub fn deduplicate(vertices: &[Vertex], indices: &[u32]) -> (Vec<Vertex>,
Vec<u32>) {
    let (vertex_count, remap) =
        generate_vertex_remap(vertices, Some(indices));
```

```
    let new_indices  = remap_index_buffer(Some(indices), vertex_count, &remap);
    let new_vertices = remap_vertex_buffer(vertices, vertex_count, &remap);
    (new_vertices, new_indices)
}
```

1.3 Vertex cache optimisation — Stage 2

MUST-HAVE

Reorders the index buffer so that if a vertex index will be seen again it appears soon after its first occurrence. Based on the Tom Forsyth algorithm with heuristics tuned across a multi-day genetic optimisation over hundreds of GPU models. Reduces vertex shader invocations 2–6x on typical meshes.

```
use meshopt::optimize_vertex_cache;

pub fn optimize_cache(indices: &mut Vec<u32>, vertex_count: usize) {
    optimize_vertex_cache(indices, vertex_count);
}

// Optional: measure quality before/after
use meshopt::analyze_vertex_cache;
let stats = analyze_vertex_cache(&indices, vertex_count, 16, 0, 0);
println!("ACMR: {:.3}  ATVR: {:.3}", stats.acmr, stats.atvr);
```

1.4 Vertex fetch optimisation — Stage 3

MUST-HAVE

After cache optimisation the vertex buffer is referenced in a new order. Reorders the vertex buffer so vertices are stored in the order they are first referenced, minimising the GPU L2 cache footprint during vertex shading.

```
use meshopt::optimize_vertex_fetch;

// Returns the new unique vertex count (unreferenced verts removed)
pub fn optimize_fetch(vertices: &mut Vec<Vertex>, indices: &mut Vec<u32>) -> usize
{
    optimize_vertex_fetch(vertices, indices)
}
```

1.5 Vertex quantisation — Stage 4

MUST-HAVE

Compact vertex attributes from 32-bit floats to hardware-optimal packed formats. A standard PBR vertex (position, normal, tangent, UV) unquantised costs 56 bytes. After quantisation: 16 bytes. 3.5x reduction in vertex bandwidth with no visible quality loss for diffuse.

```
use meshopt::{quantize_snorm, quantize_half};

#[repr(C)]
#[derive(Copy, Clone, bytemuck::Pod, bytemuck::Zeroable)]
pub struct PackedVertex {
    pub pos:      [u16; 3],   // 16-bit integer within per-mesh AABB
```

```
    pub _pad0:    u16,
    pub normal:   [i8; 2],    // oct-encoded, 16-bit total (from 96-bit)
    pub tangent:  [i8; 2],    // oct-encoded + sign bit
    pub uv:       [u16; 2],   // f16
}

pub fn pack_vertex(v: &RawVertex, aabb: &Aabb) -> PackedVertex {
    // Position: rescale to [0,65535] within AABB
    let p = (v.position - aabb.min) / aabb.extents();
    let pos = [
        (p.x * 65535.0).round() as u16,
        (p.y * 65535.0).round() as u16,
        (p.z * 65535.0).round() as u16,
    ];
    // Normal: octahedral encode
    let (nx, ny) = oct_encode(v.normal);
    let normal = [
        quantize_snorm(nx, 8) as i8,
        quantize_snorm(ny, 8) as i8,
    ];
    // UV: f16
    let uv = [
        quantize_half(v.uv.x) as u16,
        quantize_half(v.uv.y) as u16,
    ];
    PackedVertex { pos, _pad0:0, normal,
                   tangent: [0,0], uv }
}

// Decode in WGSL at runtime:
// let pos = vec3<f32>(packed.pos) / 65535.0 * mesh.bbox_scale + mesh.bbox_min;
// let normal = oct_to_vec3(vec2<f32>(packed.normal) / 127.0);
```

1.6 Mesh simplification (LOD generation) — Stage 5

MUST-HAVE

Generate LOD0 through LOD4 automatically. The error metric is world-space; convert to screen-space at runtime to eliminate pop (screen_error = world_error * proj_scale / view_dist; render this LOD if screen_error < 1px).

```
use meshopt::{simplify, SimplifyOptions};

pub struct LodLevel {
    pub indices:     Vec<u32>,
    pub world_error: f32,
}

pub fn generate_lods(indices: &[u32], vertices: &[PackedVertex]) -> Vec<LodLevel>
{
    let target_ratios = [1.0_f32, 0.5, 0.25, 0.125, 0.05];
    let mut lods = Vec::new();

    for &ratio in &target_ratios {
        let target = ((indices.len() / 3) as f32 * ratio) as usize * 3;
        let (new_idx, error) = simplify(
            indices, vertices, core::mem::size_of::<PackedVertex>(),
            target, 0.01,
```

```
            SimplifyOptions::LOCK_BORDER
        );
        lods.push(LodLevel { indices: new_idx, world_error: error });
    }
    lods
}
```

1.7 Meshlet generation — Stage 6

MUST-HAVE

Split the mesh into compact spatial clusters of ≤64 vertices and ≤128 triangles. Each meshlet uses 8-bit local indices instead of 32-bit global ones. Enables per-cluster culling in mesh shaders and is the foundation for Nanite-style cluster LOD hierarchy.

```
use meshopt::{build_meshlets, compute_meshlet_bounds};

pub struct MeshletData {
    pub meshlets:  Vec<meshopt::Meshlet>,
    pub vertices:  Vec<u32>,      // local->global index
    pub triangles: Vec<u8>,       // 8-bit local triangle indices
    pub bounds:    Vec<MeshletBounds>,
}

#[repr(C)]
#[derive(Copy, Clone, bytemuck::Pod, bytemuck::Zeroable)]
pub struct MeshletBounds {
    pub center:       [f32; 3],
    pub radius:       f32,
    pub cone_apex:    [f32; 3],
    pub cone_cutoff:  f32,        // cos(cone_half_angle)
    pub cone_axis:    [f32; 3],
    pub _pad:         f32,
}

pub fn build_meshlet_data(
    indices: &[u32],
    positions: &[[f32; 3]],
) -> MeshletData {
    let built = build_meshlets(indices, positions, 64, 128, 0.5);

    let bounds = built.meshlets.iter().map(|m| {
        let b = compute_meshlet_bounds(
            m, &built.vertices, &built.triangles, positions
        );
        MeshletBounds {
            center:      b.center,
            radius:      b.radius,
            cone_apex:   b.cone_apex,
            cone_cutoff: b.cone_cutoff,
            cone_axis:   b.cone_axis,
            _pad:        0.0,
        }
    }).collect();

    MeshletData {
        meshlets: built.meshlets,
        vertices: built.vertices,
```

```
        triangles: built.triangles,
        bounds,
    }
}
```

1.8 Two-pass compression — Stage 7

HIGH VALUE

Layer zstd on top of meshopt-encoded data. Net result: smaller than either alone, and faster total decode than zstd-only because zstd decompresses into a meshopt-compressed intermediate (much less data) and then meshopt decompresses at multi-GB/s.

```
use meshopt::{encode_index_buffer, encode_vertex_buffer};

pub fn compress_mesh(vertices: &[PackedVertex], indices: &[u32]) -> CompressedMesh
{
    let v_size  = core::mem::size_of::<PackedVertex>();

    // Stage 1: meshopt codec (spatial locality aware)
    let mo_verts  = encode_vertex_buffer(vertices, v_size).unwrap();
    let mo_idx    = encode_index_buffer(indices, vertices.len()).unwrap();

    // Stage 2: zstd on top (finds remaining statistical redundancy)
    let zv = zstd::encode_all(mo_verts.as_slice(), 3).unwrap();
    let zi = zstd::encode_all(mo_idx.as_slice(),   3).unwrap();

    CompressedMesh { vertices: zv, indices: zi,
                     vertex_count: vertices.len(),
                     index_count:  indices.len() }
}

// Runtime decode (in eustress-engine):
use meshopt::{decode_vertex_buffer, decode_index_buffer};

pub fn decompress_mesh(cm: &CompressedMesh) -> (Vec<PackedVertex>, Vec<u32>) {
    let v_size = core::mem::size_of::<PackedVertex>();
    let mo_v   = zstd::decode_all(cm.vertices.as_slice()).unwrap();
    let mo_i   = zstd::decode_all(cm.indices.as_slice()).unwrap();
    let verts  = decode_vertex_buffer::<PackedVertex>(&mo_v, cm.vertex_count,
v_size).unwrap();
    let idx    = decode_index_buffer::<u32>(&mo_i, cm.index_count).unwrap();
    (verts, idx)
}
```

1.9 Hot-reload file watcher

HIGH VALUE

Artist saves in Blender → glTF exported → baker detects change → re-bakes in <1s → Bevy AssetServer hot-reloads .emesh. The meshopt pipeline is fast enough that this round-trip is imperceptible for typical game meshes.

```
use notify::{Watcher, RecommendedWatcher, RecursiveMode, Config};

pub fn watch_and_bake(src: &Path, out: &Path) {
```

```
    let (tx, rx) = std::sync::mpsc::channel();
    let mut watcher = RecommendedWatcher::new(tx, Config::default()).unwrap();
    watcher.watch(src, RecursiveMode::Recursive).unwrap();

    for event in rx {
        match event {
            Ok(e) if e.kind.is_modify() || e.kind.is_create() => {
                for path in e.paths {
                    if path.extension().map(|e| e=="gltf"||
e=="glb").unwrap_or(false) {
                        bake_asset(&path, out);
                    }
                }
            }
            _ => {}
        }
    }
}
```

Part 2 — GPU Optimizations

2.1 Frustum culling (per-entity AABB)

MUST-HAVE

Reject entities whose AABB lies entirely outside the 6 camera frustum planes before any draw calls are submitted. Bevy 0.14 has basic frustum culling but it operates per-mesh. Extend it to work per-meshlet AABB for maximum efficiency.

```
// In crates/engine/src/rendering.rs
use bevy::render::primitives::{Aabb, Frustum};

pub fn meshlet_frustum_cull(
    frustum: &Frustum,
    center:  glam::Vec3,
    radius:  f32,
) -> bool {
    frustum.intersects_sphere(
        &bevy::render::primitives::Sphere { center: center.into(), radius },
        false  // no jitter
    )
}

// Store per-meshlet bounds in a GPU StorageBuffer:
// wgpu: BindingType::Buffer { ty: BufferBindingType::Storage { read_only:
true } }
// Compute shader reads bounds, writes draw-indirect buffer
```

2.2 GPU-driven indirect rendering

MUST-HAVE

Write draw call arguments into a GPU StorageBuffer from a compute shader. Zero CPU-GPU sync per draw call. Foundation for all further GPU-side optimisation. Required to render 100k+ object scenes without CPU bottleneck.

```
// crates/engine/src/rendering/indirect.rs

// Matches wgpu DrawIndexedIndirect layout
#[repr(C)]
#[derive(Copy, Clone, bytemuck::Pod, bytemuck::Zeroable)]
pub struct DrawIndexedIndirect {
    pub index_count:    u32,
    pub instance_count: u32,
    pub first_index:    u32,
    pub base_vertex:    i32,
    pub first_instance: u32,
}

// Build the indirect buffer (CPU side, one-time per scene load):
let indirect_buf = device.create_buffer_init(&wgpu::util::BufferInitDescriptor {
    label:    Some("indirect_draw_buf"),
    contents: bytemuck::cast_slice(&indirect_commands),
    usage:    wgpu::BufferUsages::INDIRECT | wgpu::BufferUsages::STORAGE,
});

// Dispatch from render pass:
```

```
render_pass.draw_indexed_indirect(&indirect_buf, byte_offset);

// WGSL compute shader writes draw counts via atomicAdd on a
// DrawIndirectCount buffer for multi-draw-indirect batching.
```

2.3 Depth prepass (early-Z)

MUST-HAVE

Render only depth in a fast first pass using a minimal vertex shader with no fragment output. The main pass then uses depth-equal test — fragments that would fail are discarded before the expensive PBR evaluation. Eliminates overdraw entirely for opaque geometry.

```
// In your Bevy RenderGraph, insert before the main opaque pass:
// bevy already provides Opaque3dPrepass — ensure it is enabled:

use bevy::core_pipeline::prepass::DepthPrepass;

// Add to camera entity:
commands.entity(camera).insert(DepthPrepass);

// Verify your materials implement the prepass pipeline:
// impl Material for MyMaterial {
//     fn prepass_vertex_shader() -> ShaderRef { ... }
//     fn prepass_fragment_shader() -> ShaderRef { ShaderRef::Default }
// }
```

2.4 GPU instancing for repeated meshes

MUST-HAVE

Draw thousands of trees, rocks, and props in a single draw call by writing per-instance transform data into a StorageBuffer. 10–1000x fewer draw calls. Combine with frustum culling compute pass that only writes visible instances to the instance buffer.

```
#[repr(C)]
#[derive(Copy, Clone, bytemuck::Pod, bytemuck::Zeroable)]
pub struct InstanceData {
    pub transform: [[f32; 4]; 4],  // row-major Mat4
    pub aabb_min:  [f32; 3],
    pub _pad0:     f32,
    pub aabb_max:  [f32; 3],
    pub _pad1:     f32,
}

// In Bevy: use bevy::render::render_resource::StorageBuffer
// Upload all instance data once; culling compute writes
// visible subset into a second 'visible_instances' buffer.
// Bind visible_instances as vertex buffer slot 1 in mesh shader.
```

2.5 Per-meshlet back-face cone culling (WGSL)

MUST-HAVE

Reject entire meshlets whose geometry is back-facing from the current camera. Cone data is pre-computed by meshoptimizer. Typical rejection: 30–50% of meshlets on organic characters. This runs in a compute pre-pass before any mesh shader dispatch.

```
// crates/engine/src/shaders/meshlet_cull.wgsl

struct MeshletBounds {
    center:      vec3<f32>,
    radius:      f32,
    cone_apex:   vec3<f32>,
    cone_cutoff: f32,
    cone_axis:   vec3<f32>,
    _pad:        f32,
};

@compute @workgroup_size(64)
fn cull_meshlets(
    @builtin(global_invocation_id) id: vec3<u32>,
) {
    let i = id.x;
    if i >= arrayLength(&meshlet_bounds) { return; }
    let b = meshlet_bounds[i];
    let cam = camera.position.xyz;

    // Frustum sphere test
    if !sphere_in_frustum(b.center, b.radius) { return; }

    // Backface cone test
    let apex_to_cam = normalize(cam - b.cone_apex);
    if dot(apex_to_cam, b.cone_axis) < b.cone_cutoff { return; }

    // Visible: append draw command
    let slot = atomicAdd(&draw_count, 1u);
    visible_draws[slot] = build_draw_indirect(i);
}
```

2.6 Clustered forward lighting

HIGH VALUE

Divide the view frustum into a 3D grid of tiles (e.g. 16x9x24). Each tile stores the indices of lights that overlap it. Fragment shaders only iterate the lights for their tile. Handles 1000+ dynamic lights at near-constant cost.

```
// crates/engine/src/rendering/clustered_light.rs

pub const CLUSTER_X: u32 = 16;
pub const CLUSTER_Y: u32 = 9;
pub const CLUSTER_Z: u32 = 24;  // exponential depth slices
pub const MAX_LIGHTS_PER_CLUSTER: u32 = 64;

#[repr(C)]
#[derive(Copy, Clone, bytemuck::Pod, bytemuck::Zeroable)]
pub struct ClusterEntry {
    pub light_offset: u32,  // into light_indices[]
    pub light_count:  u32,
}
```

```
// Compute pass: for each light, find overlapping clusters,
// atomicAdd into per-cluster counters, write light indices.
// Fragment shader samples: cluster = get_cluster(frag_coord, depth);
//   for i in 0..clusters[cluster].light_count {
//       let li = light_indices[clusters[cluster].light_offset + i];
//       color += evaluate_light(lights[li], ...);
//   }

// bevy_pbr already has clustered lights — verify it is enabled in
// your CameraPlugin and not overridden by custom render pipeline.
```

Part 3 — CPU / ECS Optimizations

3.1 Switch global allocator to mimalloc

MUST-HAVE

One line change. mimalloc is a high-performance allocator that reduces fragmentation and improves throughput in multithreaded workloads by 10–30% in Bevy benchmarks. Critical for archetype moves, asset loading, and parallel system spawning.

```
# crates/engine/Cargo.toml
[dependencies]
mimalloc = { version = "0.1", default-features = false }

// crates/engine/src/main.rs  — must be at crate root
#[global_allocator]
static ALLOC: mimalloc::MiMalloc = mimalloc::MiMalloc;
```

3.2 Audit system parallelism — remove false dependencies

MUST-HAVE

Bevy runs independent systems in parallel automatically, but a mutable resource access blocks all other systems that touch that resource. Audit every system signature: change ResMut to Res wherever you only read, and split large resource types.

```
// BAD: blocks all other systems touching GameConfig while running
fn my_system(mut config: ResMut<GameConfig>, query: Query<&Transform>) { }

// GOOD: read-only, allows full parallelism
fn my_system(config: Res<GameConfig>, query: Query<&Transform>) { }

// Visualise the system graph to find bottlenecks:
# Cargo.toml
bevy_mod_debugdump = "0.10"

// main.rs
bevy_mod_debugdump::print_schedule_graph(&mut app, Update);
// Open the output .dot file in https://dreampuf.github.io/GraphvizOnline
```

3.3 Change-detection filters on hot queries

MUST-HAVE

Use Changed<T> and Added<T> query filters to skip idle entities. A system that iterates 10,000 entities looking for dirty transforms becomes effectively free when only 30 actually moved. This is the single highest-leverage ECS optimisation in Bevy.

```
// Only process entities whose Transform actually changed this frame
fn sync_physics(
    query: Query<(&Transform, &mut RigidBodyHandle), Changed<Transform>>
) {
    for (transform, mut body) in &query {
        // Only runs for entities that moved
        update_physics_body(&transform, &mut body);
    }
}
```

```
// Combine with Added<T> to handle newly spawned entities:
fn init_new_entities(
    query: Query<(Entity, &Mesh), Added<Mesh>>
) { /* only runs for newly inserted Mesh components */ }

// EventReader: process input only when events actually fire
fn handle_input(mut events: EventReader<PlayerInput>) {
    for event in events.read() { /* ... */ }
}
```

3.4 Parallel iteration with par_iter()

MUST-HAVE

Replace sequential for loops over large queries with Rayon-backed parallel iterators. Bevy's par_iter() automatically distributes work across the TaskPool thread count. Most impactful on transform propagation, AI processing, and physics broadphase.

```
use bevy::prelude::*;

fn propagate_transforms(
    query: Query<(&GlobalTransform, &mut CachedWorldPos)>,
) {
    // Sequential — bad for 10k+ entities:
    // for (gt, mut cache) in &mut query { ... }

    // Parallel — distributes across all CPU cores:
    query.par_iter_mut().for_each(|(gt, mut cache)| {
        cache.0 = gt.translation();
    });
}

// Configure TaskPool thread count in app startup:
app.add_plugins(TaskPoolPlugin {
    task_pool_options: TaskPoolOptions {
        compute: TaskPoolThreadAssignmentPolicy {
            min_threads: 1,
            max_threads: usize::MAX,
            percent: 0.75,  // 75% of cores for compute
        },
        ..default()
    },
});
```

3.5 Pipelined rendering (frame overlap)

MUST-HAVE

Simulate frame N+1 on the CPU while the GPU renders frame N. Hides the entire CPU extraction cost. Bevy 0.14 supports this via PipelinedRendering — it is off by default and costs one line to enable.

```
// main.rs
use bevy::render::pipelined_rendering::PipelinedRenderingPlugin;

app.add_plugins(PipelinedRenderingPlugin);
```

```
// Warning: extracted resources are one frame behind the main world.
// Ensure your rendering systems read from ExtractedXxx resources,
// not from the main world resources directly.
// Bevy handles this correctly for built-in components.
// Custom extraction: implement ExtractComponent or ExtractResource.
```

3.6 Component storage tuning

HIGH VALUE

Bevy stores components in two layouts: Table (default, cache-friendly for iteration) and SparseSet (fast add/remove, slow iteration). Wrong choice = 5-10x iteration slowdown. Rule: hot-iterated components use Table; frequently added/removed components use SparseSet.

```
// Default (Table) — correct for components iterated every frame:
#[derive(Component)]
pub struct Velocity(pub glam::Vec3);

// SparseSet — correct for components that come and go:
#[derive(Component)]
#[component(storage = "SparseSet")]
pub struct Stunned { pub until: f32 }

#[derive(Component)]
#[component(storage = "SparseSet")]
pub struct Selected;  // editor selection marker

#[derive(Component)]
#[component(storage = "SparseSet")]
pub struct Sleeping;  // physics sleep state
```

3.7 Per-frame arena allocator (bumpalo)

HIGH VALUE

Use bump allocators for per-frame scratch data: culling result lists, sort key arrays, visibility sets. Zero deallocation cost — just reset the bump pointer at frame end. Eliminates per-frame heap pressure from temporary Vec allocations.

```
# Cargo.toml
bumpalo = "3"

// In your frame loop / render extract system:
use bumpalo::Bump;

#[derive(Resource)]
pub struct FrameArena(pub Bump);

// Reset at start of each frame:
fn reset_arena(mut arena: ResMut<FrameArena>) {
    arena.0.reset();
}

// Use for scratch allocations in render systems:
fn cull_meshlets(arena: Res<FrameArena>, ...) {
```

```
    let visible: &mut Vec<u32, _> =
        bumpalo::collections::Vec::new_in(&arena.0);
    // ... fill visible list without heap allocation
}
```

Part 4 — World State Optimizations

4.1 Dynamic BVH for spatial queries

MUST-HAVE

Maintain a Bounding Volume Hierarchy over all entity AABBs. Enables O(log n) frustum queries, raycasts, and broad-phase physics overlap tests in one structure. Update incrementally on Changed<Transform> events — don't rebuild every frame.

```
# Cargo.toml
parry3d = "0.17"   # ships with Rapier, includes BVH (Dbvt)
# or use the standalone bvh crate for a simpler interface
bvh = "0.9"

use bvh::bvh::Bvh;
use bvh::aabb::{Aabb, Bounded};
use bvh::ray::Ray;

#[derive(Resource)]
pub struct WorldBvh {
    pub bvh:      Bvh<f32, 3>,
    pub shapes:   Vec<EntityShape>,
}

pub struct EntityShape {
    pub entity: Entity,
    pub aabb:   Aabb<f32, 3>,
}

// Rebuild BVH only when transforms changed:
fn rebuild_bvh(
    query: Query<(Entity, &GlobalTransform, &Aabb), Changed<GlobalTransform>>,
    mut bvh_res: ResMut<WorldBvh>,
) {
    // Incremental: remove/re-insert moved entities only
    // Full rebuild when >30% of entities moved (heuristic):
    bvh_res.bvh = Bvh::build(&mut bvh_res.shapes);
}
```

4.2 Fixed timestep simulation

MUST-HAVE

Run physics, game logic, and networking on a fixed tick rate (e.g. 64Hz) independent of the render frame rate. Prevents spiral-of-death on slow frames and is a prerequisite for deterministic multiplayer reconciliation. Bevy 0.14 has this built in.

```
use bevy::time::Fixed;

// Configure the fixed timestep rate:
app.insert_resource(Time::<Fixed>::from_hz(64.0));

// Add physics/gameplay systems to FixedUpdate schedule:
app.add_systems(FixedUpdate, (
    physics_step,
    ai_tick,
```

```
    network_send,
).chain());

// Render at variable rate; interpolate between fixed snapshots:
// Store prev_transform alongside Transform component.
// In Update: lerp prev_transform → transform by overstep fraction.
fn interpolate_render(
    time: Res<Time<Fixed>>,
    mut query: Query<(&Transform, &PrevTransform, &mut RenderTransform)>
) {
    let t = time.overstep_fraction();
    for (cur, prev, mut render) in &mut query {
        render.0 = Transform {
            translation: prev.0.translation.lerp(cur.translation, t),
            rotation:    prev.0.rotation.slerp(cur.rotation, t),
            scale:       cur.scale,
        };
    }
}
```

4.3 Entity sleep / activation radius

MUST-HAVE

Physics and AI are expensive. Only simulate entities within an activation radius of any player. Put Rapier rigid bodies to sleep when below velocity threshold. Typical result: 60–80% physics CPU reduction in a populated open world.

```
use bevy_rapier3d::prelude::*;

const ACTIVATION_RADIUS: f32 = 200.0;

fn manage_sleep(
    players: Query<&GlobalTransform, With<Player>>,
    mut bodies: Query<(
        &GlobalTransform,
        &mut RigidBody,
        &mut Sleeping,
    )>,
) {
    for (body_tf, mut rb, mut sleeping) in &mut bodies {
        let near_any_player = players.iter().any(|p| {
            p.translation().distance(body_tf.translation()) < ACTIVATION_RADIUS
        });
        if near_any_player {
            sleeping.sleeping = false;
        } else {
            sleeping.sleeping = true;
        }
    }
}
```

4.4 Distance-based component LOD

HIGH VALUE

Reduce update frequency or skip heavy systems entirely for distant entities. AI state machines, animation blend trees, and audio processing are unnecessary at 500m. Gate expensive systems with a distance-bucketed marker component.

```
#[derive(Component)]
pub enum SimLod { Full, Reduced, Minimal, Dormant }

fn assign_sim_lod(
    players: Query<&GlobalTransform, With<Player>>,
    mut entities: Query<(&GlobalTransform, &mut SimLod), Without<Player>>,
) {
    for (tf, mut lod) in &mut entities {
        let dist = players.iter()
            .map(|p| p.translation().distance(tf.translation()))
            .fold(f32::INFINITY, f32::min);
        *lod = match dist {
            d if d <  50.0  => SimLod::Full,
            d if d < 200.0  => SimLod::Reduced,
            d if d < 500.0  => SimLod::Minimal,
            _               => SimLod::Dormant,
        };
    }
}

// Gate systems by LOD:
fn ai_full(query: Query<&mut AiState, With<SimLod::Full>>) { }
fn ai_reduced(query: Query<&mut AiState, With<SimLod::Reduced>>) { }
// SimLod::Dormant entities are never touched by AI systems
```

4.5 World cell streaming

HIGH VALUE

Divide the world into 2D grid cells. Load/unload ECS entities as the camera moves. Each cell is a Bevy DynamicScene serialised to disk. Load ahead of the player asynchronously using the IoTaskPool so there are no frame hitches.

```
use bevy::scene::DynamicScene;

#[derive(Resource)]
pub struct WorldGrid {
    pub cell_size:    f32,         // e.g. 500.0 metres
    pub loaded_cells: HashMap<(i32,i32), Handle<DynamicScene>>,
}

fn stream_world(
    player:      Query<&GlobalTransform, With<Player>>,
    mut grid:    ResMut<WorldGrid>,
    asset_server: Res<AssetServer>,
    mut commands: Commands,
) {
    let pos = player.single().translation();
    let cx  = (pos.x / grid.cell_size).floor() as i32;
    let cz  = (pos.z / grid.cell_size).floor() as i32;

    // Load cells within 2-cell radius:
    for dx in -2..=2_i32 { for dz in -2..=2_i32 {
        let key = (cx+dx, cz+dz);
```

```
        if !grid.loaded_cells.contains_key(&key) {
            let path = format!("world/cell_{}_{}.scn.ron", key.0, key.1);
            let h = asset_server.load(&path);
            grid.loaded_cells.insert(key, h);
        }
    }}
    // Unload cells beyond 3-cell radius (omitted for brevity)
}
```

4.6 Network delta compression (bevy-webtransport)

HIGH VALUE

Only replicate component values that changed since the last acknowledged frame. Use a bitmask per entity. Can reduce bandwidth 10–100x versus naive full-state replication. Combine with priority scheduling: send high-priority entities first within a byte budget.

```
// crates/engine/src/networking/replication.rs

#[derive(Default)]
pub struct ReplicationState {
    /// last snapshot sent to each peer, keyed by peer_id
    pub snapshots: HashMap<PeerId, HashMap<Entity, ComponentSnapshot>>,
}

pub fn build_delta_packet(
    state:    &ReplicationState,
    current:  &HashMap<Entity, ComponentSnapshot>,
    peer:     PeerId,
    budget:   usize,  // max bytes this tick
) -> DeltaPacket {
    let last = state.snapshots.get(&peer);
    let mut packet = DeltaPacket::new();
    let mut used = 0;

    // Priority: screen coverage * time_since_last_update
    let mut entities: Vec<_> = current.iter().collect();
    entities.sort_by(|a,b| priority(b).partial_cmp(&priority(a)).unwrap());

    for (entity, snap) in entities {
        if used >= budget { break; }
        if last.map(|l| l.get(entity)) != Some(Some(snap)) {
            let delta = snap.diff(last.and_then(|l| l.get(entity)));
            used += delta.byte_len();
            packet.push(*entity, delta);
        }
    }
    packet
}
```

Part 5 — Radiance Cascades GI

Radiance Cascades (Alexander Sannikov, 2024) is identified as the superior GI technique for EustressEngine — no light leaking, no distance ceiling, scales from small rooms to open world. It is implemented as a series of wgpu compute passes added to the Bevy RenderGraph.

Why Radiance Cascades beats Lumen

Lumen uses signed distance fields and screen-space tracing with a two-level probe structure. Radiance Cascades uses a hierarchical cascade of flat 2D radiance intervals — each cascade level covers a larger spatial range at coarser angular resolution. The merging pass propagates light up the cascade chain with no leaking artifacts and no per-probe memory explosion. It degrades gracefully on low-end hardware by simply reducing the cascade count.

5.1 Cascade data structure

MUST-HAVE

```
// crates/engine/src/gi/radiance_cascades.rs

pub const CASCADE_COUNT: usize = 6;
pub const BASE_PROBE_SPACING: f32 = 1.0;   // metres, level 0
pub const BASE_RAY_COUNT:     u32  = 8;     // rays per probe, level 0

// Each cascade level doubles spacing and quadruples rays
// Level k: spacing = BASE * 4^k,  rays = BASE_RAYS * 4^k

#[derive(Resource)]
pub struct RadianceCascades {
    /// One texture per cascade level: RG11B10 radiance, 3D (X,Y,Z_slice)
    pub textures:       [wgpu::Texture; CASCADE_COUNT],
    pub texture_views:  [wgpu::TextureView; CASCADE_COUNT],
    pub probe_pipeline: wgpu::ComputePipeline,  // trace rays
    pub merge_pipeline: wgpu::ComputePipeline,  // merge level k+1 → k
    pub bind_groups:    [wgpu::BindGroup; CASCADE_COUNT],
}

impl RadianceCascades {
    pub fn probe_count(&self, level: usize, world_size: glam::Vec2) -> glam::UVec2
{
        let spacing = BASE_PROBE_SPACING * 4.0_f32.powi(level as i32);
        (world_size / spacing).ceil().as_uvec2()
    }
    pub fn ray_count(&self, level: usize) -> u32 {
        BASE_RAY_COUNT * 4_u32.pow(level as u32)
    }
}
```

5.2 Probe radiance tracing pass (WGSL)

MUST-HAVE

```
// crates/engine/src/shaders/rc_trace.wgsl

@compute @workgroup_size(8, 8)
fn trace_probes(
```

```
    @builtin(global_invocation_id) id: vec3<u32>,
) {
    let level   = push.cascade_level;
    let spacing = BASE_PROBE_SPACING * pow(4.0, f32(level));
    let rays    = BASE_RAY_COUNT     * (1u << (2u * level));

    let probe_xy = id.xy;
    let probe_pos = (vec2<f32>(probe_xy) + 0.5) * spacing;

    var radiance_sum = vec3<f32>(0.0);

    for (var r = 0u; r < rays; r++) {
        // Fibonacci sphere distribution for ray direction
        let dir = fibonacci_direction(r, rays);

        // Interval: [spacing/2, spacing*2] — skip near-field
        let t_min = spacing * 0.5;
        let t_max = spacing * 2.0;

        let hit = trace_ray(probe_pos, dir, t_min, t_max);
        radiance_sum += hit.radiance;
    }

    // Store average radiance for this probe
    let avg = radiance_sum / f32(rays);
    textureStore(cascade_tex, vec2<i32>(probe_xy), encode_rg11b10(avg));
}
```

5.3 Merge pass — propagate up cascade chain (WGSL)

MUST-HAVE

```
// crates/engine/src/shaders/rc_merge.wgsl
// Runs once per cascade level pair (k+1 → k), from coarsest to finest

@compute @workgroup_size(8, 8)
fn merge_cascades(
    @builtin(global_invocation_id) id: vec3<u32>,
) {
    let probe_xy = id.xy;
    // Level k radiance (fine)
    var fine = decode_rg11b10(
        textureLoad(cascade_fine, vec2<i32>(probe_xy), 0).xy
    );

    // Bilinearly sample the coarser level k+1 at this probe's world pos
    let world_pos   = (vec2<f32>(probe_xy) + 0.5) * push.fine_spacing;
    let coarse_uv   = world_pos / push.coarse_spacing;
    let coarse_rad  = textureSampleLevel(cascade_coarse, linear_clamp,
                                         coarse_uv, 0.0).rgb;

    // Merge: fine sees its local interval; coarse fills the rest
    let merged = fine + coarse_rad * (1.0 - fine_coverage(fine));
    textureStore(cascade_fine, vec2<i32>(probe_xy), vec4<f32>(merged, 1.0));
}

// fine_coverage estimates how much light was captured in the fine trace.
```

```
// Simple version: saturate(luminance(fine) / expected_max_luminance)
```

5.4 Bevy RenderGraph integration

MUST-HAVE

```
// crates/engine/src/gi/mod.rs
use bevy::render::render_graph::{RenderGraph, RenderLabel};

#[derive(Debug, Hash, PartialEq, Eq, Clone, RenderLabel)]
pub enum GiLabel {
    RcTrace,   // trace all cascade levels
    RcMerge,   // merge coarse → fine
}

pub fn add_gi_passes(render_graph: &mut RenderGraph) {
    render_graph.add_node(GiLabel::RcTrace, RcTraceNode::new());
    render_graph.add_node(GiLabel::RcMerge, RcMergeNode::new());

    // GI must complete before the main lighting pass reads it
    render_graph.add_node_edge(GiLabel::RcMerge, bevy::core_pipeline::
        core_3d::graph::Core3d::MainOpaquePass);

    // Trace runs before merge
    render_graph.add_node_edge(GiLabel::RcTrace, GiLabel::RcMerge);
}

// In the main lighting WGSL, sample level-0 probes for indirect diffuse:
// let probe_uv  = world_pos.xz / (BASE_PROBE_SPACING * probe_tex_size);
// let irradiance = textureSampleLevel(cascade_0, linear_clamp, probe_uv,
0.0).rgb;
// color += albedo * irradiance * INV_PI;
```

Part 6 — Complete Cargo Workspace Setup

6.1 Root workspace Cargo.toml

```
# eustress/Cargo.toml
[workspace]
resolver = "2"
members = [
  "crates/common",
  "crates/engine",
  "crates/client",
  "crates/baker",
  "crates/bevy-camera-controller",
  "crates/bevy-webtransport",
]

[workspace.dependencies]
bevy      = { version = "0.14", features = ["dynamic_linking"] }
glam      = "0.27"
bytemuck  = { version = "1", features = ["derive"] }
serde     = { version = "1", features = ["derive"] }
bincode   = "1"

[profile.dev]
opt-level = 1

[profile.dev.package."*"]
opt-level = 3  # optimize deps even in dev builds

[profile.release]
opt-level = 3
lto       = "thin"
codegen-units = 1
```

6.2 Engine crate dependencies

```
# crates/engine/Cargo.toml
[dependencies]
bevy       = { workspace = true }
mimalloc   = { version = "0.1", default-features = false }
meshopt    = { version = "0.3", default-features = false, features = ["decode"] }
zstd       = "0.13"
parry3d    = "0.17"
rapier3d   = { package = "bevy_rapier3d", version = "0.27" }
glam       = { workspace = true }
bytemuck   = { workspace = true }
bumpalo    = "3"
dashmap    = "6"
crossbeam  = "0.8"

[[bin]]
name = "eustress-engine"
path = "src/main.rs"
```

6.3 Baker crate dependencies

```
# crates/baker/Cargo.toml
[dependencies]
meshopt  = "0.3"
gltf     = "1.4"
zstd     = "0.13"
notify   = "6"
glam     = { workspace = true }
bytemuck = { workspace = true }
serde    = { workspace = true }
bincode  = { workspace = true }
rayon    = "1"   # parallel asset baking

[[bin]]
name = "eustress-baker"
path = "src/main.rs"
```

6.4 Recommended implementation order

DO THESE FIRST

- 1.1 Add eustress-baker to workspace
- 3.1 Switch to mimalloc (one-liner)
- 1.2 → 1.5 Baker pipeline: deduplicate → cache → fetch → quantise → LOD
- 3.3 Change-detection filters on all hot queries
- 3.4 par_iter() on transform propagation
- 3.5 PipelinedRendering plugin
- 2.3 Depth prepass (DepthPrepass component on camera)
- 2.1 + 2.5 Frustum cull + meshlet cone cull compute pass
- 2.2 GPU indirect rendering
- 1.7 Meshlet generation in baker
- 4.2 FixedUpdate simulation schedule
- 4.3 Entity sleep radius
- 5.1 → 5.4 Radiance Cascades GI (largest undertaking — 2–4 weeks)
- 1.8 Two-pass mesh compression
- 4.4 Component LOD bucketing
- 4.5 World cell streaming
- 4.6 Network delta compression

Reference: vk_lod_clusters

For the Nanite-style cluster LOD hierarchy (Section 1.7 extended), see github.com/nickmqb/vk_lod_clusters — Nvidia's Vulkan sample that combines meshoptimizer cluster data with a full streaming LOD runtime. The Rust port of clusterlod.h is the correct starting point for the eustress-baker LOD hierarchy build.